

Iniziato	venerdì, 3 luglio 2026, 09:07
Stato	Completato
Terminato	venerdì, 3 luglio 2026, 10:37
Tempo impiegato	1 ora 29 min.
Valutazione	15,79 su un massimo di 18,00 (87,7%)

Informazione

[ARES simulator / Simulatore ARES](#)

Domanda 1

Risposta errata

Punteggio ottenuto 0,00 su 1,00

Considerando un processore RISC-V a 32 bit, si supponga che il registro PC abbia il valore **0x00400004** (in esadecimale), e che l'istruzione corrente sia la prima istruzione riportata nel frammento di codice RISC-V di seguito.

Determinare l'indirizzo che sarà presente nel PC dopo l'esecuzione dell'istruzione **blt t0, zero, cont1**

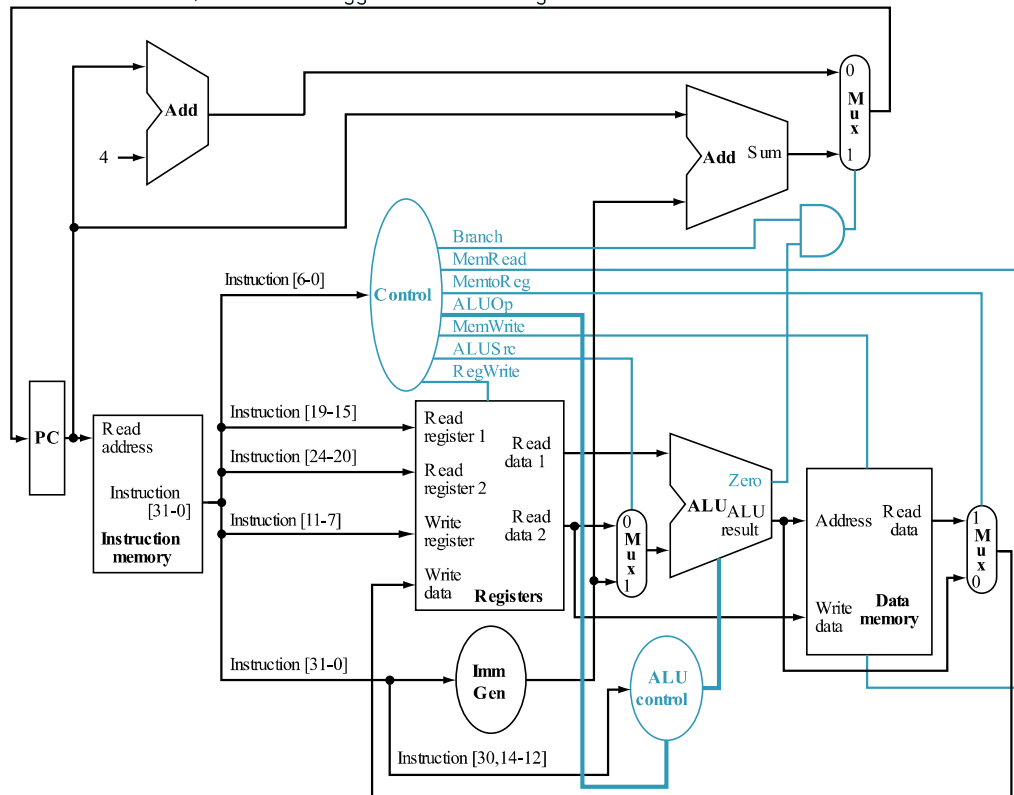
```

addi t0, zero, -4
addi t1, zero, 4
sll t0, t0, t1
blt t0, zero, cont1
addi t0, zero, 0
jal zero, cont0

cont1:
    addi t0, zero, 1

cont0:
    nop
  
```

Come riferimento, il PC viene aggiornato dalla seguente rete nel RISC-V:



Scegli un'alternativa:

- ☐ a. 0x00400010
- ☐ b. 0x00400018
- ☐ c. 0x00400020
- ☐ d. 0x0040001C
- ☒ e. 0x00400014 ✖

La risposta corretta è: **0x0040001C**

Domanda **2**

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Il codice sottostante rappresenta una funzione RISC-V (RV32I). Cosa fa?

```
mystery: slli a1, a1, 2
         slli a2, a2, 2
         add t0, a0, a1
         add t1, a0, a2
         lw  t2, 0(t0)
         lw  t3, 0(t1)
         sw  t3, 0(t0)
         sw  t2, 0(t1)
         ret
```

Scegli un'alternativa:

- ☐ a. Legge le due parole selezionate in t2 e t3, lasciando invariata la memoria.
- ☐ b. Scambia i due caratteri alle posizioni a1 e a2 di una stringa puntata da a0.
- ☐ c. Scambia i byte agli indirizzi a0+a1 e a0+a2 in memoria.
- ☐ d. Copia l'elemento di indice a2 in quello di indice a1, lasciando invariato l'indice a2.
- ☒ e. Scambia tra loro gli elementi di indici a1 e a2 dell'array puntato da a0. ✓
- ☐ f. Scambia il contenuto dei registri a1 e a2.

La risposta corretta è: Scambia tra loro gli elementi di indici a1 e a2 dell'array puntato da a0.

Domanda 3

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Si consideri un processore RISC-V (RV32I) con memoria indirizzabile al byte. La memoria principale contiene inizialmente i valori riportati nella figura, dove il numero a sinistra di ciascuna cella rappresenta il suo indirizzo.

Memoria

7	0x87
6	0x86
5	0x85
4	0x84
3	0x83
2	0x82
1	0x81
0	0x80

Il processore esegue le seguenti due istruzioni, nell'ordine:

```
sh x6, 2(x5)
lw x7, 0(x5)
```

Prima dell'esecuzione, il registro x5 = **0x00000000** e il registro x6 = **0x1234ABCD**.

Quale valore (4 byte, in esadecimale) contiene il registro x7 dopo l'esecuzione di entrambe le istruzioni? Riportare il risultato in esadecimale, senza spazi (ad esempio: 0xFFFFFFFF):

 ✓

Domanda 4

Parzialmente corretta

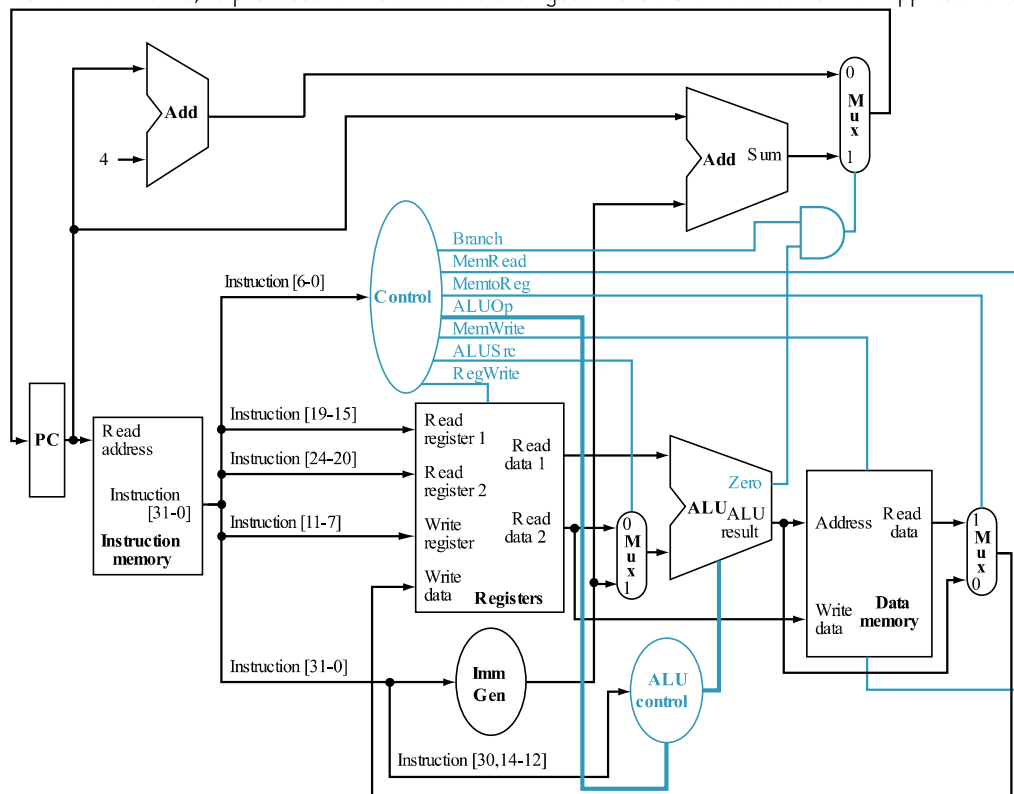
Punteggio ottenuto 0,80 su 1,00

Si consideri che il processore RISC-V a ciclo singolo visto durante le lezioni abbia appena prelevato la seguente istruzione: or x10,x11,x12.

Quali sono i valori delle uscite dell'unità di controllo per questa istruzione?

1. ALUSrc: ✗
2. RegWrite: ✓
3. MemRead: ✓
4. MemWrite: ✓
5. Branch: ✓

Come riferimento, il processore RISC-V a ciclo singolo visto durante le lezioni è rappresentato dal seguente diagramma:



Domanda 5

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Si considerino tre diversi processori P1, P2 e P3 che eseguono lo stesso insieme di istruzioni.

- P1 ha una frequenza di clock di 2.5 GHz e un CPI (Cicli Per Istruzione) di 1.0.
- P2 ha una frequenza di clock di 2.0 GHz e un CPI di 3.0.
- P3 ha una frequenza di clock di 1.0 GHz e un CPI di 1.0.
- Quale processore ha le prestazioni migliori espresse in numero di istruzioni al secondo? ✓
- Si consideri un programma costituito da $5.0 \cdot 10^9$ istruzioni. Determinare il tempo (in secondi) richiesto da P3 per eseguire il programma. Nota: esprimere il risultato usando la virgola , come separatore tra la parte intera e quella frazionaria se necessario. Non inserire alcun carattere di spaziatura o unità di misura. ✓

Domanda 6

Parzialmente corretta

Punteggio ottenuto 0,75 su 1,00

Determinare se sono vere o false le seguenti affermazioni:

- I bus asincroni hanno un "master clock" per controllare la lunghezza dei cicli del bus. ✓
- L'uso della cache è basato sul principio della località del codice, i.e., il fatto che i programmi accedono alla memoria casualmente. ✗
- Una memoria cache di maggiore dimensione può migliorare la prestazione, ma aumenta il costo. ✓
- I bus "stretti" sono più economici di quelli larghi offrendo comunque una banda di trasferimento più larga. ✓

Domanda 7

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Due compilatori diversi hanno tradotto il seguente frammento di codice C:

```
if (!(a && b)) {  
    x = 1;  
}
```

Assumi che **a** sia in **t0**, **b** sia in **t1**, e **x** sia in **t2**.

Compilatore A:

```
1      and t3, t0, t1  
2      beqz t3, xset  
3      j skip  
4 xset: li t2, 1  
5 skip: nop
```

Compilatore B (incompleto):

```
1      _____  
2      beqz t1, xset  
3      j skip  
4 xset: li t2, 1  
5 skip: nop
```

Quale istruzione manca nel codice del Compilatore B (linea 1) per renderlo logicamente equivalente a quello del Compilatore A?

Scegli un'alternativa:

- ☐ a. bnez t0, xset
- ☒ b. beqz t0, xset ✓
- ☐ c. beqz t1, xset
- ☐ d. bnez t0, skip
- ☐ e. beq t0, t1, xset
- ☐ f. and t3, t0, t1

La risposta corretta è: beqz t0, xset

Domanda 8

Parzialmente corretta

Punteggio ottenuto 0,64 su 1,00

Si consideri il seguente frammento di programma RISC-V (RV32I). I registri sono indicati con il loro **numero** (es. **x6**).

```
lw x19, 0(x30)
beq x19, x13, REG_EQ
addi x20, x20, 4
slli x19, x19, 1
REG_EQ: and x29, x6, x18
       xori x13, x6, 16
```

a) (0.5 punti) Qual è il valore (in decimale) dell'offset che l'assemblatore calcola per l'istruzione **beq**? ✓

b) (0.5 punti) Codifica in linguaggio macchina l'istruzione **beq** nel **formato B**, organizzato nei seguenti campi (il bit 31 è il più significativo, il bit 0 il meno significativo):

31	30:25	24:20	19:15	14:12	11:8	7	6:0
imm[12]	imm[10:5]	rs2	rs1	funct3	imm[4:1]	imm[11]	opcode

Per **beq** sono dati: opcode = **1100011**, funct3 = **000**. Riporta i bit di ciascun campo come stringa di bit, **senza spazi**:

imm[12]: ✗ imm[10:5]: ✗ rs2: ✓ rs1: ✗ funct3: **000** imm[4:1]:
 ✗ imm[11]: ✓ opcode: **1100011**

Domanda 9

Parzialmente corretta

Punteggio ottenuto 0,60 su 1,00

Si supponga di avere una cache a **mappatura diretta** per un sistema con memoria indirizzabile al byte e indirizzi a 32 bit. Si supponga di voler implementare una cache con 1 KiB di dati, e che ogni blocco contenga 8 byte.

- (0.2 punti) Quante linee ha la cache? ✓
- (0.2 punti) Quanti bit sono necessari per indirizzare le linee? ✓
- (0.2 punti) Qual è la dimensione del tag in bit? ✓
- (0.4 punti) Se si accede all'indirizzo 0x00000429, in quale linea della cache verrà caricato il blocco contenente questo indirizzo? ✗

Domanda **10**

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Quale delle seguenti affermazioni è vera riguardo alla differenza tra RV32I (architettura a 32 bit) e RV64I (architettura a 64 bit)?

Scegli un'alternativa:

- ☐ a. RV64I richiede il doppio della larghezza di banda della memoria per il fetch delle istruzioni
- ☐ b. RV64I utilizza codifica delle istruzioni a 64 bit mentre RV32I utilizza codifica a 32 bit
- ☐ c. RV64I utilizza opcode diversi per le operazioni aritmetiche rispetto a RV32I
- ☒ d. RV64I opera su registri a 64 bit, mentre RV32I opera su registri a 32 bit ✓
- ☐ e. RV64I ha il doppio del numero di registri general-purpose rispetto a RV32I

La risposta corretta è: RV64I opera su registri a 64 bit, mentre RV32I opera su registri a 32 bit

Domanda **11**

Risposta corretta

Punteggio ottenuto 4,00 su 4,00

Scrivi una procedura chiamata `find_first_digit(char *str)` che cerca il primo carattere cifra (da '0' a '9') in una stringa e restituisce la sua posizione (indice). Se non viene trovata nessuna cifra, la funzione restituisce -1.

Ad esempio, se `str` contiene "hello123world", la funzione restituisce 5 (la posizione del primo '1'). Se `str` contiene "no digits here", la funzione restituisce -1.

Il seguente codice in C implementa `find_first_digit` (convertilo in RISC-V):

```
int find_first_digit(char *str) {
    int i = 0;
    while (str[i] != '\0') {
        if (str[i] >= '0' && str[i] <= '9') {
            return i;
        }
        i++;
    }
    return -1;
}
```

Si ricorda che il codice ASCII del carattere '0' è 48 e '9' è 57, e che la pseudo-istruzione `li t0, '0'` carica il valore 48 in `t0`.

Attenzione:

- Incollare solo la funzione `find_first_digit` (in RISC-V) nel campo sottostante
- Attenzione alle convenzioni di chiamata!
- Usare il seguente codice `main` per lo sviluppo e il debugging nel simulatore

```
.globl _start
.data
    str: .string "hello123world"

.text
_start:
    # chiama find_first_digit
    la  a0, str
    jal ra, find_first_digit

    li  a7, 10
    ecall

#####
# completa la funzione find_first_digit nel campo sottostante
```

Risposta (Criteri di penalizzazione: 0 %)Ripristina risposta

L'editor Ace non è pronto. Prova a ricaricare la pagina?
Falling back to raw text area.

```

# carico il primo carattere
# se il carattere blt 48 -> salta
# se il carattere bgt 57 -> salta
# altrimenti, move in a0 il valore di i

find_first_digit:

li t0, 0                # i = 0
li t5, '0'              # t6 = 48
li t4, '9'              # t5 = 57

loop:

lbu t1, 0(a0)           # carica il primo byte in t1

beqz t1, ritorna_meno_1  # se la stringa è terminata -> ritorna_meno_1

bge t1, t5, check_2      # char >= 48

incrementa:

addi t0, t0, 1           # i++
addi a0, a0, 1           # punta al prossimo carattere

j loop

check_2:

ble t1, t4, return_index # char <= 57
j incrementa

return_index:

mv a0, t0                # in a0 restituisci l'index corrente
jr ra

```

	Test	Atteso	Ottenuto	
✓	.section .data str: .string "hello123world"	a0: 5	a0: 5	✓

Tutti i test sono stati superati ✓

► Visualizza/Nascondi soluzione dell'autore (Asm)

Risposta corretta

Punteggio di questo invio: 4,00/4,00.

Commento:

OK

Domanda **12**

Risposta corretta

Punteggio ottenuto 4,00 su 4,00

Scrivere una procedura chiamata `count_prime(int *arr, int n)` che riceve un array di interi positivi `arr` e la sua dimensione `n`, e restituisce il numero di numeri primi presenti nell'array.

La funzione `count_prime` deve utilizzare la funzione `isprime` che riceve un intero e restituisce un numero positivo se il numero è primo, 0 altrimenti. Soluzioni che non utilizzano la funzione `isprime` saranno considerate errate.

`isprime` è già implementata in RISC-V e non deve essere riscritta. Una possibile implementazione di `isprime` è fornita di seguito, ma il codice dovrebbe funzionare con qualsiasi altra implementazione purché segua le stesse convenzioni di chiamata.

Il valore di ritorno deve essere lasciato nel registro `a0`.

Il seguente codice in C implementa `count_prime` (convertirlo in RISC-V):

```
// int (in C) è equivalente a word (in RISC-V)
int count_prime(int *arr, int n) {
    int count = 0;
    for (int i = 0; i < n; i++) {
        if (isprime(arr[i]) != 0) {
            count++;
        }
    }
    return count;
}
```

Ad esempio, se `arr = [2, 3, 4, 5, 6, 7, 8, 9, 10, 11]`, la funzione restituisce 5 perché i numeri primi sono: 2, 3, 5, 7, 11.

Attenzione:

- Incollare solo la funzione `count_prime` (in RISC-V) nel campo sottostante
- Prestare attenzione alle convenzioni di chiamata!
- Utilizzare il seguente codice `main` per lo sviluppo e il debug nel simulatore

```
.globl _start
.data
    arr:    .word 2, 3, 4, 5, 6, 7, 8, 9, 10, 11
    n:      .word 10

.text
_start:
    # chiama count_prime
    la  a0, arr
    la  a1, n
    lw  a1, 0(a1)
    jal ra, count_prime

    # exit
    li  a7, 10
    ecall

#####
# isprime
# a0 -> numero da controllare
# restituisce numero positivo se primo, 0 se non primo
#####
isprime:
    mv  t0, a0
    addi a0, zero, 0
    li  t1, 2
    blt t0, t1, label_a
label_b:
    bge t1, t0, label_c
    rem t3, t0, t1
    beq t3, zero, label_a
    addi t1, t1, 1
    j   label_b
label_c:
    li  a0, -1
    srli a0, a0, 1
label_a:
    ret

#####
# completa la funzione count_prime qui sotto
#####
```

Risposta (Criteri di penalizzazione: 0 %)

Ripristina risposta

L'editor Ace non è pronto. Prova a ricaricare la pagina?
Falling back to raw text area.

```

li s0, 0          # i = 0
li s1, 0          # count = 0
mv s2, a1         # s2 = n
mv s3, a0         # s3 = l'indirizzo di a0

loop:

bge s0, s2, return_count

lw a0, 0(s3)      # primo valore dell array nel regsitro che richiede la funzione
jal ra, isprime   # chiamo la funzione, in a0 mi restituisce 1 se il numero era primo, 0 se non
era un numero primo

beqz a0, incrementa # if (a0 = 1) -> count++
                    # altrimenti, punta al prossimo numero
j count

incrementa:

addi s0, s0, 1    # i++
addi s3, s3, 4    # punta al prossimo carattere

j loop

count:

addi s1, s1, 1    # count++

j incrementa

return_count:

mv a0, s1

lw ra, 0(sp)
lw s0, 4(sp)
lw s1, 8(sp)
lw s2, 12(sp)
lw s3, 16(sp)

addi sp, sp, 32

```

	Test	Atteso	Ottenuto	
✓	<pre> .section .data arr: .word 2, 3, 4, 5, 6, 7, 8, 9, 10, 11 n: .word 10 </pre>	a0: 5	a0: 5	✓

Tutti i test sono stati superati ✓

► Visualizza/Nascondi soluzione dell'autore (Asm)

Risposta corretta

Punteggio di questo invio: 4,00/4,00.

Commento:

OK